request, decide whether it's allowed, and enforce the outcome—the way each component is implemented can vary widely depending on the scale and maturity of the system.

Table 5.4 shows how the core components of the architecture—PEP, PDP, PAP, and PIP—are generally realized in an RBAC model. In smaller systems, these components are often closely integrated with the application that enforces access, evaluates permissions, and retrieves role assignments within the same codebase. In larger or more mature environments, RBAC responsibilities may be externalized into shared libraries or centralized services, or integrated with enterprise identity systems.

Table 5.4 Mapping RBAC onto the authorization reference architecture. Each architectural component plays a unique role in authorization. This table illustrates how RBAC systems generally implement each component, along with notes on common variations.

Component	What it does	How RBAC implements it
Policy Enforcement Point (PEP)	Intercepts access requests and enforces the decision	Usually embedded in application code, API gateway, or middleware
Policy Decision Point (PDP)	Determines whether the requested action is allowed	Often, a function or module that checks whether the user's role includes the required permission; may be embedded, shared as a library, or centralized as a service
Policy Administration Point (PAP)	Manages and stores roles, permissions, and assignments	Roles and mappings stored in a database, LDAP/AD directory, config files, or managed through an admin UI or an infrastructure-as-code (IaC) system like Terraform
Policy Information Point (PIP)	Supplies user and role data to the PDP at runtime	Role data often comes from identity provider, database lookup, or session context

RBAC's simplicity and architectural versatility aren't just a convenience—they're one of the reasons RBAC has endured. By cleanly separating concerns, RBAC enables the evolution from tightly coupled, application-level logic to enterprise-grade, externalized authorization services without altering the core model. Whether permissions are enforced inline or roles are defined using infrastructure as code, the RBAC pattern remains recognizable, adaptable, and effective.